1. What is LangChain?

LangChain is a framework for building LLM applications.

It helps connect:

- LLMs
- Prompts
- Memory
- Tools
- RAG
- Databases
- APIs

without writing everything from scratch.

Basic Architecture

```
User
 ↓
Prompt Template
 ↓
LLM
 ↓
Output Parser
 ↓
Response
```

Simple LangChain Example

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

prompt = ChatPromptTemplate.from_template(
    "Explain {topic} in simple words"
)

chain = prompt | llm

response = chain.invoke({
    "topic": "Machine Learning"
})

print(response.content)
```

Components of LangChain

1. Prompt Templates

```
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic}"
)

print(prompt.format(topic="AI"))
```

---

2. LLM Models

OpenAI

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="gpt-4o-mini"
)
```

HuggingFace

```
from langchain_huggingface import HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id="mistralai/Mistral-7B-Instruct-v0.3"
)
```

---

3. Output Parsers

```
from langchain_core.output_parsers import StrOutputParser

chain = prompt | llm | StrOutputParser()

result = chain.invoke({
    "topic":"Python"
})

print(result)
```

---

4. Memory

Conversation Memory

```
from langchain.memory import ConversationBufferMemory
```

```
memory = ConversationBufferMemory()

memory.save_context(
    {"input":"Hi"},
    {"output":"Hello"}
)
```

---

5. Chains

Sequential Chain

```
chain = prompt | llm
```

Multiple Chains

```
summary_chain = summary_prompt | llm

quiz_chain = quiz_prompt | llm
```

LangChain RAG

Most common use case.

```
PDF
 ↓
Chunks
 ↓
Embeddings
 ↓
Vector DB
 ↓
Retriever
 ↓
LLM
 ↓
Answer
```

RAG Example

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

loader = PyPDFLoader("book.pdf")
docs = loader.load()

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200
)
```

```
chunks = splitter.split_documents(docs)
```

Embeddings

```
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

FAISS

```
from langchain_community.vectorstores import FAISS

db = FAISS.from_documents(
    chunks,
    embeddings
)
```

Retriever

```
retriever = db.as_retriever()
```

Query

```
docs = retriever.invoke(
    "What is neural network?"
)

print(docs)
```

Tools in LangChain

Example Calculator Tool

```
from langchain.tools import tool

@tool
def add(a:int,b:int):
    return a+b
```

Agent in LangChain

```
from langchain.agents import initialize_agent
```

```
agent = initialize_agent(
    tools,
    llm,
    agent="zero-shot-react-description"
)
```

Problems with LangChain Agents

Suppose Agent:

```
Tool A
 ↓
Tool B
 ↓
Tool C
```

What if:

- Tool B fails?
- Need retry?
- Need branching?
- Human approval?
- Multi-agent?

LangChain becomes difficult.

This is where LangGraph comes.

What is LangGraph?

LangGraph is a framework for building:

- AI Agents
- Multi-Agent Systems
- Human-in-the-loop
- Stateful Workflows
- Complex Decision Trees

using Graphs.

LangGraph Architecture

```
Node
 ↓
Node
```

```
  ↓
Node
```

Each node:

- LLM
- Tool
- Agent
- Function

can be connected.

LangGraph Concepts

1. State

State stores information.

```
from typing import TypedDict

class State(TypedDict):
    question:str
    answer:str
```

---

2. Nodes

```
def chatbot(state):
    return {
        "answer":"Hello"
    }
```

---

3. Edges

```
graph.add_edge(
    "chatbot",
    END
)
```

---

4. Graph

```
graph.compile()
```

LangGraph Example

```
from typing import TypedDict
from langgraph.graph import StateGraph

class State(TypedDict):
    question:str
    answer:str

def chatbot(state):
    return {
        "answer":
        f"Answering {state['question']}"
    }

graph = StateGraph(State)

graph.add_node(
    "chatbot",
    chatbot
)

graph.set_entry_point(
    "chatbot"
)

app = graph.compile()

result = app.invoke({
    "question":"What is AI?"
})

print(result)
```

Conditional Routing

```
Question
   ↓
Programming?
 /     \
Yes    No
 ↓      ↓
Code   General
```

Example:

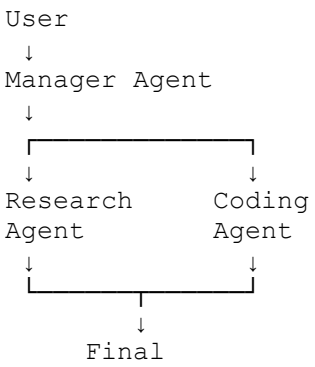
```
def router(state):

    if "python" in state["question"]:
        return "coding"

    return "general"
```

```
graph.add_conditional_edges(
    "router",
    router
)
```

Multi-Agent System



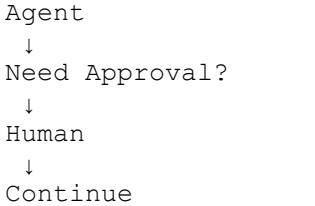
Example:

```
graph.add_node(
    "research_agent",
    research_agent
)

graph.add_node(
    "coding_agent",
    coding_agent
)

graph.add_node(
    "manager",
    manager
)
```

Human-in-the-Loop



```
from langgraph.types import interrupt

interrupt("Approve?")
```

Checkpointing

Agent remembers state.

```
from langgraph.checkpoint.memory import MemorySaver

memory = MemorySaver()
```

Persistence

```
graph.compile(
    checkpointer=memory
)
```

Tool Calling in LangGraph

```
from langgraph.prebuilt import ToolNode
```

```
tool_node = ToolNode(
    [search_tool]
)
```

LangGraph Agent Example

```
from langgraph.prebuilt import create_react_agent

agent = create_react_agent(
    llm,
    tools
)
```

LangChain vs LangGraph

Feature	LangChain	LangGraph
Prompting	□	□
Chains	□	□
RAG	□	□
Tools	□	□
Agents	□	□
Multi-Agent	Limited	Excellent
Human Approval	Difficult	Native
Workflow Control	Basic	Advanced
State Management	Limited	Excellent
Branching	Difficult	Native
Long Running Tasks	Poor	Excellent
Production Agents	Moderate	Excellent

Same Problem in Both

LangChain

```
chain = prompt | llm
```

Simple.

LangGraph

```
graph.add_node()
graph.add_edge()
graph.compile()
```

More code.

When to Use LangChain?

Use LangChain if:

Chatbot

Resume Analyzer

PDF Q&A

RAG Application

Streamlit AI App

Career Assistant

FAQ Bot

Example:

```
Resume Analyzer
↓
Prompt
↓
LLM
↓
Result
```

No graph needed.

When to Use LangGraph?

Use LangGraph if:

AI Agent

Multi-Agent System

Autonomous Agent

Research Agent

Coding Agent

Planning Agent

Human Approval Workflow

Tool Routing

Customer Support Agent

Example:

```
User
↓
Planner
↓
Research Agent
↓
Tool Calls
↓
Review Agent
↓
Human Approval
↓
Final Response
```

Interview Questions

What is LangChain?

Framework for building LLM applications using prompts, chains, memory, tools, and RAG.

What is LangGraph?

Framework for building stateful AI agents using graph-based workflows.

Why was LangGraph introduced?

To overcome limitations of traditional LangChain agents by providing:

- State management
- Multi-agent support
- Conditional routing
- Human-in-the-loop
- Long-running workflows